

Takween Tutors — Web Platform

Consolidate the RBAC/dashboard foundation and close the confirmed gaps on the live takweentutors.com platform (React/Vite + Express/Prisma/Supabase)

Prepared for	Haseeb Sajjad
Project type	Production build — ongoing development on the live Takween Tutors web platform
Reference product	takweentutors.com — live, already-launched platform (not a trial/demo)
Stack	React 18 + Vite + TanStack Query + Tailwind (apps/web) · Express + TypeScript (apps/api) · PostgreSQL via Prisma, hosted on Supabase · Supabase Auth · Stripe Checkout · Zoom Server-to-Server OAuth
Core feature set	Four role-based dashboards (Parent, Student, Tutor, Admin) — bookings, 1:1 and group (1:5/1:10) sessions, Zoom access, progress reports, payments
Distribution (QA phase)	Staging/preview deploy with a dedicated review login per role, ahead of promotion to production

Purpose

Takween Tutors is the live tutoring platform for students aged 4–18 across 40+ subjects at Primary, GCSE, A-Level, 11+, and BTEC level, connecting students with tutors for live Zoom sessions while parents manage bookings and payments. Unlike the Flutter mobile projects, this is not a from-scratch or trial build — it is an already-launched product carrying real bookings, and the work here is about consolidating and extending it responsibly.

The platform has grown feature-by-feature under time pressure, and a few structural gaps have built up alongside it: role and permission logic is duplicated across the frontend and API instead of living in one place, Supabase Row-Level Security is only applied to the newest tables (Group, GroupEnrolment, GroupSession) rather than the other twenty or so tables that rely solely on the Express API's own checks, and several "not built" items flagged directly by the business against the live app (queues, alerts, linked report pages) are still open. This brief formalises that work — the RBAC/schema foundation, the confirmed dashboard gaps, and the Group Sessions build — into one build-and-review cycle, with the same discipline around graceful error handling and reviewable submissions used on the mobile side.

This is real, live infrastructure with real payments and Zoom credentials behind it. Ship changes to a staging/preview environment first, and get sign-off before anything touches production.

Design & UI — Extend the Existing System, Don't Reinvent It

The visual language, navigation structure, and content already exist on the live site — this is consolidation and gap-closing, not a redesign. Where the plan calls for a single nav/permission registry to replace the current per-role `if` blocks, or a shared `<Guard role={...}>` to replace the two

overlapping route-guard components, keep the resulting UI visually consistent with what's already shipped rather than introducing a new pattern.

Where the business-verified flow diagrams describe behaviour that doesn't match the live codebase, treat the flow diagrams as the source of truth for intended behaviour — but flag anything ambiguous rather than guessing.

What to Build

1. RBAC & Navigation Consolidation

- Merge `ProtectedRoute` and `RoleRoute` into one guard (component or hook) with a single "access denied" state and a single redirect rule for unauthenticated users
- Replace the hardcoded per-role `if` blocks driving dashboard navigation with one nav/permission registry, so Parent, Student, Tutor, and Admin nav items are all defined in one place
- Bring API-side role checks onto one consistent convention (currently gated once at router mount for Admin, but per-route everywhere else)

2. Supabase Schema & RLS Hardening

- Extend Row-Level Security beyond `Group`/`GroupEnrolment`/`GroupSession` to the tables currently relying only on API-level checks (`User`, `Session`, `Message`, `BlockPurchase`, `SessionReport`, and others)
- Retire the stale `snake_case` RLS definitions (`supabase/rls_fix.sql`, `supabase/migration.sql`) that target a schema Prisma has already superseded
- Apply the already-written, not-yet-applied DB hardening plan: soft-delete, FK onDelete policy, indexes, and storing money as pence rather than decimal
- Add a persisted `Notification` model — today `NotificationContext` is in-memory client-side toasts only, so nothing survives a refresh or reaches a second device

3. Admin Dashboard — Closing Operational Gaps

- Missing-reports queue, safeguarding alerts, and renewal nudges — the operationally highest-leverage items flagged against the live app
- A surface for recording retention/storage status (see Group Sessions below)

4. Parent / Student / Tutor Dashboard Fixes

- Link the existing (but currently orphaned) parent and student report pages into their respective nav — they work but aren't reachable today
- Close out the remaining confirmed "not built" gaps from the flow-diagram audit

5. Group Sessions (1:5 / 1:10) — Idea 1: Admin Assigns

- Data model for a Group ("batch"): fixed tutor, one year-group band (max 2 school years wide), a fixed session count matching the buyer's block size (4 or 10), and a seat cap (5 or 10) enforced at the database level so a batch can never be oversold
- On purchase: auto-place the child into a batch with free seats that hasn't started; otherwise present the parent a dashboard choice — after payment, never at checkout — between joining a running

batch (with recordings for missed sessions) or waiting for the next one, defaulting to "wait" if no choice is made within 72 hours

- One Zoom meeting per session (not per batch), created only when needed (session 1 at publish, the rest on demand or in bulk, admin's choice), with the join link opening on the day of the session — same pattern as the existing 1:1 flow
- Recordings are screen-and-audio only, gated on full-batch consent (if any family in a batch withholds consent, that batch isn't recorded for anyone), with checkout consent wording that reflects recordings may be shared with students joining later
- Admin can move a child between groups, with a parent notification; attendance is marked by the tutor, with Zoom data treated only as a hint, not the record of truth

6. Zoom Production Configuration

- Wire the production Server-to-Server OAuth credentials and licensed host account into live settings, and run an end-to-end test that a real meeting can be created, joined, and (for groups) recorded per the consent rules above

Handling Missing Permissions, Partial Data, and Failed Requests

- A user without the right role for a page must see one consistent "access denied" state — never a crash, blank screen, or silently-broken layout
- Once RLS is extended, a denied database read must fail the same way a denied API call does today — no case where the two disagree
- If the API is unreachable or a request fails, dashboards should show the last-loaded data or a clear retry state, not a blank screen — consistent with the standard already set for the mobile apps
- Notification delivery failures (email, in-app) must not block the underlying action (booking, group assignment, payment) from completing

Technical Requirements

- Changes land in `apps/web` (React/Vite/TanStack Query/Tailwind) and `apps/api` (Express/TypeScript) as appropriate; database changes go through Prisma migrations against Supabase Postgres
- No API keys, service-role keys, or Zoom/Stripe secrets committed to GitHub — environment variables / config exclusion, same rule as on all Takween projects
- Every schema or RLS change ships as a reviewed Prisma migration with a rollback path — no direct edits against the production database
- Seat limits and other business-critical constraints (batch size, year-group band width, session-count matching) are enforced at the database level, not only in application code
- Code is on GitHub, organised sensibly, with a reasonable commit history reflecting incremental work, not a single end-of-cycle commit
- Changes are verified on a staging/preview deploy, with typecheck (`npm run typecheck`) passing, before any production promotion
- No unnecessary personal data collected or stored beyond what each role's dashboard genuinely requires

If a business decision is genuinely blocking (for example, a Zoom account/meeting-capacity limit, an ambiguous consent rule, or a flow-diagram item that doesn't match live behaviour and isn't yours to resolve), flag it early with what you attempted — don't spend excessive time working around it before raising it. Same rule as on the mobile projects.

What to Submit

When you're ready to submit a phase of this work, complete a Submission for Review document with the fields below.

#	Field	What to include
1	Staging/preview access	URL, plus a working login for each of the four roles (Parent, Student, Tutor, Admin)
2	Scope covered	Which Phase 9 sub-phases and/or Group Sessions items this submission covers, and what's explicitly deferred
3	RBAC & RLS coverage	Which route guards were merged, which tables now have RLS, and which (if any) still rely on API-only checks, with reasoning
4	GitHub repository	Full URL, plus a link to recent commits or a brief summary of what each covers
5	Migrations applied	List of Prisma migrations in this submission and the rollback plan for each
6	Loom screen recording	4–6 minute video showing: nav/permissions per role, the admin queues/alerts built, the group-session assign and choice-screen flow, a Zoom join, and behaviour on denied access or a failed request
7	What I Built	5 to 10 lines in plain English — what changed from a user's perspective
8	Problems encountered	Any issues hit during the build and how you resolved them — be honest and specific
9	How to test	Step-by-step instructions, including seed data or test accounts needed for each role
10	Known limitations	Anything that doesn't work perfectly, or that you'd improve with more time
11	Path to Production Answer	Your full answer to the plan below, written in paragraphs
12	Submission date	The date you're submitting this phase

QA Review Criteria

#	Assessment Point
1	Is there one nav/permission registry and one route guard, replacing the previous duplicated logic?
2	Does RLS now cover the tables it's supposed to, matching the API's own role checks rather than contradicting them?

3	Do the admin queues/alerts and the linked parent/student report pages actually surface, and reflect real data?
4	Does the group-session assignment and choice-screen flow match the confirmed decisions (seat caps, year-band width, session-count matching, consent-gated recording)?
5	Are seat/capacity limits enforced by the database, not just the UI?
6	Does the platform handle denied access and failed requests gracefully, without crashing or showing blank/broken data?
7	Is the code on GitHub and sensibly organised, with reviewed migrations and a reasonable commit history?
8	Was this verified on staging before any production promotion?
9	Was the Submission for Review document completed in full?

Path to Production

Write up your plan for the points below in your submission — plain English, a few short paragraphs, not bullet points.

What to Cover

Right now, RLS and route-guard consolidation are being rolled out table by table and page by page. What would need to be true before you'd call the platform's permission model fully closed — no table left relying on API-only checks, no page reachable through a stale guard — and how would you verify that, rather than assuming it from the pages you've personally touched?

Group Sessions here is Idea 1 (admin manually assigns batches). The business's own long-term pick for scale is Idea 3 (system-generated groups). What in this build would carry over directly, and what would need to be re-architected to get there once volume justifies it?

Numan's Flutter apps consume some of this same platform (auth, dashboard data, Zoom access). Where should the contract between the web API and the mobile apps be made explicit now, so a schema or auth change here doesn't silently break the mobile side later?

Final Note

The two most important things in this project are correctness of the permission model — a parent should never be able to see another family's data, whether through the UI, a stale route guard, or a table with no RLS — and graceful handling of denied access, missing data, or a failed request. This is live infrastructure with real bookings and payments behind it: get these right, verify on staging, and the rest follows.